# JS

# Introduction to JavaScript

Student Exercise Guide · 80-minute session

## Part 1 — Introduction Notes

### 1.1 What is JavaScript?

JavaScript (JS) is a lightweight, interpreted programming language primarily used to make web pages interactive. It was created in 1995 by Brendan Eich at Netscape in just 10 days, and it has since grown into one of the most widely used programming languages in the world.

Today, JavaScript runs:

- In every web browser (Chrome, Firefox, Safari, Edge) — no installation required
- On servers and the command line, using Node.js
- In mobile apps, desktop apps, games, and even hardware devices

**Why does JavaScript matter?**

• It is the only programming language that browsers understand natively.
• It allows you to build the interactive parts of websites — buttons, animations, forms, live updates.
• Learning JS unlocks both front-end (what users see) and back-end (server logic) development.
• JavaScript is consistently ranked #1 in developer surveys worldwide.

### 1.2 How JavaScript Fits Into the Web

Every website you visit is built with three core technologies that work together:

| HTML | CSS | JavaScript |
|---|---|---|
| Structure | Presentation | Behaviour |
| | The skin. Colors, fonts, layout, visual style. | The muscles. Makes things move, respond, and change dynamically. |

| | | |
|---|---|---|
| The skeleton. Defines headings, paragraphs, images, links. | | |

## 1.3 Why Learn JavaScript First?

Many beginner programmers wonder which language to start with. JavaScript is an excellent first language because:

- Instant feedback — you run it directly in your browser. No compilation, no setup.
- Visual results — your code immediately changes something you can see on screen.
- Forgiving syntax — small mistakes often still run, helping you learn by experimentation.
- Huge community — millions of tutorials, Stack Overflow answers, and open-source libraries.
- One language, many uses — the same language works in your browser AND on a server.

**Real-world examples of JavaScript in action**

• The Like button on Instagram — a click triggers JavaScript to update a counter.
• Google Search suggestions — as you type, JavaScript fetches results in real time.
• Online shopping carts — items are added/removed without reloading the page.
• Maps and navigation — Google Maps is almost entirely JavaScript.

## 1.4 Setting Up Your Environment

For today's exercises you only need two things:

- A modern browser — Chrome or Firefox recommended.
- The browser console — Press `F12` (Windows/Linux) or `Cmd+Option+I` (Mac), then click the Console tab.

Alternatively, you can write code in a free online editor like:

- JSFiddle — jsfiddle.net
- CodePen — codepen.io
- Replit — replit.com

**Remember**

The console is your best friend as a JavaScript learner. It is where you see the output of your code, read error messages, and test ideas instantly. Never be afraid to type something and press Enter.

# Part 2 — Exercise Blocks

Each block below includes fully written code with line-by-line explanations. Read the explanation first, then type the code yourself — typing it (not copy-pasting) is an important part of learning.

## Block 1 The Browser Console — 10 minutes

### What you will learn

How to use your browser's built-in JavaScript console to run code immediately and see results. This is the fastest way to experiment with JavaScript.

### Exercise 1.1 — Your first line of code

Type the following in your browser console and press Enter:

```
console.log("Hello, world!");
```

**What does this do?** The function `console.log()` prints a value to the console so you can see it. It is the JavaScript equivalent of 'print' in other languages. The text inside the quotes is called a **string** — a sequence of characters.

**Why do we need it?** JavaScript normally runs silently in the background. Without `console.log()`, you would have no way to see what your code is doing. Every developer uses it constantly for testing.

### Exercise 1.2 — Basic arithmetic

JavaScript can work like a calculator. Type each of these one at a time:

```
2 + 2       // addition  → 4
10 * 5      // multiplication  → 50
100 / 4     // division  → 25
7 - 3       // subtraction  → 4
2 ** 8      // exponentiation (2 to the power of 8)  → 256
17 % 5      // modulus (remainder of 17 ÷ 5)  → 2
```

**Why do we need arithmetic?** Calculations are at the heart of almost every program — prices in a shopping cart, scores in a game, pixels in an animation. The `%` (modulus) operator might

look unusual but it is very useful: it tells you the remainder after division, which is perfect for tasks like checking whether a number is even or odd.

> **Tip**
> Everything after // on a line is a comment. JavaScript ignores it. Comments are notes for the human reading the code. Always write comments to explain WHY your code does what it does.

## Block 2 Variables and Data Types

15 minutes

### What you will learn

How to store values in named containers (variables) and understand the three most common data types: strings, numbers, and booleans.

### Exercise 2.1 — Declaring variables

Type this code in your console or editor, one line at a time:

```
// Declare a variable that can change later
let age = 20;
console.log(age);          // Output: 20

// Declare a variable that will never change
const name = "Alex";
console.log(name);         // Output: Alex

// Try to reassign the constant — this will give an error
const name = "Jordan";    // TypeError: Assignment to constant variable
```

**What does this do?** Variables are labelled boxes that hold data. `let` creates a box whose contents you can swap out. `const` creates a box that is sealed — once a value goes in, it cannot be replaced. The error on the last line is intentional and important: errors teach you the rules.

**Why do we need variables?** Without variables, you would have to hard-code every value and you could never reuse a piece of data. Imagine writing a username 50 times in your code — then the user changes their name. With a variable, you update it in one place.

### Exercise 2.2 — Data types

JavaScript has several built-in types. Here are the three you will use most as a beginner:

```
// Number — any numeric value (integer or decimal)
let score = 100;
let temperature = 36.6;

// String — text, always wrapped in quotes
let greeting = "Hello, world!";
let city     = 'Skopje';      // single quotes work too

// Boolean — only two possible values: true or false
let isLoggedIn = true;
let hasWon     = false;

// Check the type of any value with typeof
console.log(typeof score);      // 'number'
console.log(typeof greeting);   // 'string'
console.log(typeof isLoggedIn); // 'boolean'
```

**Why do types matter?** JavaScript behaves differently depending on the type of a value. Adding two numbers gives a sum. Adding two strings joins them (called concatenation). Adding a number to a string can produce unexpected results. Understanding types prevents bugs.

## Exercise 2.3 — String operations

Strings are not just static text — you can manipulate them:

```
let firstName = "Alex";
let lastName  = "Johnson";

// Concatenation (joining strings with +)
let fullName = firstName + " " + lastName;
console.log(fullName);      // "Alex Johnson"

// Template literals (the modern way — use backticks)
let message = `Hello, ${firstName}! You have ${3 + 4} messages.`;
console.log(message);       // "Hello, Alex! You have 7 messages."

// Useful string properties and methods
console.log(fullName.length);           // 12
console.log(fullName.toUpperCase());    // "ALEX JOHNSON"
console.log(fullName.includes("Alex")); // true
```

**What are template literals?** The backtick syntax `` `Hello, ${firstName}` `` lets you embed variables and expressions directly inside a string using `${}`. This is much cleaner than string concatenation with `+`. Prefer template literals in your own code.

## Block 3 Functions

20 minutes

### What you will learn

How to group code into reusable named blocks called functions. Functions are the most fundamental building block of any program.

**Why do we need functions?**

Imagine you need to greet a user in 10 different places in your program. Without functions, you copy the same code 10 times. If you then want to change the greeting, you have to find and update all 10 copies.

A function lets you write the code once, give it a name, and then call that name wherever you need it. Change the function once and every place that uses it is updated automatically.

### Exercise 3.1 — Your first function

```
// Function declaration
function greet(name) {
  return "Hello, " + name + "!";
}

// Calling the function
console.log(greet("Taylor"));    // "Hello, Taylor!"
console.log(greet("Sam"));       // "Hello, Sam!"
console.log(greet("the class")); // "Hello, the class!"
```

**Breaking it down:**

| | |
|---|---|
| `function greet` | Declares a function named greet. |
| `(name)` | A parameter — a placeholder that receives a value when the function is called. |
| `{ ... }` | The function body — curly braces wrap all the code that runs when the function is called. |

| | |
|---|---|
| `return` | Sends a value back to wherever the function was called from. |
| `greet("Taylor")` | Calling the function with "Taylor" as the argument. It substitutes name = "Taylor" inside the body. |

### Exercise 3.2 — Arrow functions

Arrow functions are a shorter way to write functions, introduced in ES6 (2015). They are widely used in modern JavaScript.

```
// Traditional function declaration
function double(n) {
  return n * 2;
}

// Same function as an arrow function
const double = (n) => n * 2;

// Both are called the same way
console.log(double(5));   // 10
console.log(double(21));  // 42

// Arrow function with multiple parameters
const add = (a, b) => a + b;
console.log(add(3, 7));   // 10

// Arrow function with a body (multiple lines)
const describe = (name, age) => {
  const sentence = `${name} is ${age} years old.`;
  return sentence;
};
console.log(describe("Alex", 20));  // "Alex is 20 years old."
```

**When to use which?** Both work. Arrow functions are preferred in modern code because they are shorter and have slightly simpler rules. Use function declarations for top-level named functions, and arrow functions for shorter inline logic.

### Exercise 3.3 — Challenge: isEven

Write a function that returns true if a number is even, and false if it is odd. Try writing it yourself before reading the solution below.

```
// The % (modulus) operator returns the remainder of division
```

```
// Even numbers have no remainder when divided by 2

const isEven = (n) => n % 2 === 0;

console.log(isEven(4));   // true
console.log(isEven(7));   // false
console.log(isEven(100)); // true
console.log(isEven(0));   // true  (0 is considered even)
```

**What is ===?** In JavaScript, `===` is the strict equality operator. It checks that both the value AND the type are the same. Use `===` instead of `==` to avoid subtle bugs caused by JavaScript's automatic type conversion.

## Block 4 Conditionals and Loops

15 minutes

### What you will learn

How to make your code take different paths based on conditions (if/else), and how to repeat actions automatically (for loops). These two features together make programs powerful.

### Exercise 4.1 — if / else

```
function classify(number) {
  if (number > 0) {
    return "positive";
  } else if (number < 0) {
    return "negative";
  } else {
    return "zero";
  }
}

console.log(classify(42));   // 'positive'
console.log(classify(-7));   // 'negative'
console.log(classify(0));    // 'zero'
```

**How it works:** JavaScript evaluates each condition from top to bottom and runs the first block whose condition is `true`. If no condition matches, the `else` block runs as a fallback. You can have as many `else if` branches as you need.

## Exercise 4.2 — for loop

```
// Log numbers 1 through 10
for (let i = 1; i <= 10; i++) {
  console.log(i);
}

// Breaking down the loop header:
// let i = 1    → start: i begins at 1
// i <= 10      → condition: keep looping while i is 10 or less
// i++          → update: add 1 to i after each iteration
```

**Why do we need loops?** Imagine you need to print numbers 1 to 1000. Writing 1000 separate `console.log()` calls would be absurd. A loop does the same task any number of times automatically. Loops are also essential for processing lists of data — every item in a shopping cart, every row in a database result.

## Exercise 4.3 — Combining conditionals and loops

```
// Loop from 1 to 20 and label each number
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) {
    console.log(i + " is even");
  } else {
    console.log(i + " is odd");
  }
}

// Using break to stop early
for (let i = 1; i <= 20; i++) {
  if (i === 15) {
    console.log("Stopping at 15!");
    break;   // exits the loop immediately
  }
  console.log(i);
}
```

**What is break?** `break` immediately exits the loop, skipping any remaining iterations. It is useful when you find what you are looking for and do not need to check the rest. Its counterpart, `continue`, skips the current iteration and moves to the next one.

## Block 5 Mini Challenge: FizzBuzz

15 minutes

### What you will learn

How to combine everything: functions, conditionals, loops, and the modulus operator to solve a classic programming problem. FizzBuzz is one of the most famous interview questions in software development.

**The FizzBuzz rules**

Write a function that, given a number n:

- Returns 'FizzBuzz' if n is divisible by both 3 and 5
- Returns 'Fizz' if n is divisible only by 3
- Returns 'Buzz' if n is divisible only by 5
- Returns the number itself if none of the above apply

Then loop from 1 to 30 and print the result for each number.

### Step-by-step solution

Try writing the solution yourself first. When you are ready, compare with the code below:

```
function fizzBuzz(n) {
  // IMPORTANT: check the combined case first!
  // If you check % 3 first, you might return 'Fizz' for a FizzBuzz number
  if (n % 3 === 0 && n % 5 === 0) {
    return "FizzBuzz";
  } else if (n % 3 === 0) {
    return "Fizz";
  } else if (n % 5 === 0) {
    return "Buzz";
  } else {
    return n;   // just the number
  }
}

// Loop 1-30 and print each result
for (let i = 1; i <= 30; i++) {
  console.log(fizzBuzz(i));
}

// Expected output excerpt:
// 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz ...
```

**Why check FizzBuzz first?** If you check `n % 3 === 0` before checking the combined case, then for n = 15 (which is divisible by both), you would return `"Fizz"` and never reach the FizzBuzz condition. Order of conditions matters!

**What is &&?** The `&&` operator means AND — both conditions must be true for the whole expression to be true. Its partner, `||`, means OR — at least one condition must be true.

### Bonus challenge — add 'Whizz'

Extend fizzBuzz so that multiples of 7 return 'Whizz' (and handle combinations like 21, 35, 105). Try it on your own!

```
function fizzBuzzWhizz(n) {
  let result = '';
  if (n % 3 === 0) result += 'Fizz';
  if (n % 5 === 0) result += 'Buzz';
  if (n % 7 === 0) result += 'Whizz';
  return result || n;  // if result is empty, return the number
}

console.log(fizzBuzzWhizz(15));   // 'FizzBuzz'
console.log(fizzBuzzWhizz(21));   // 'FizzWhizz'
console.log(fizzBuzzWhizz(105));  // 'FizzBuzzWhizz'
console.log(fizzBuzzWhizz(11));   // 11
```

**A smarter approach:** Instead of nested if/else, this version builds up a result string by concatenating parts. This is more elegant because adding a new rule (like Whizz) only requires adding one line, and all combinations are handled automatically. `result || n` returns `result` if it has content, otherwise returns `n` — this is called short-circuit evaluation.

## Block 6 Wrap-Up and Reflection — 5 minutes

### What you covered today

In 80 minutes you have gone from zero to writing real, working JavaScript. Here is a summary of everything you learned:

| | |
|---|---|
| `console.log()` | Print values to the console to see what your code is doing. |

| | |
|---|---|
| `let / const` | Declare variables. Use const by default; use let when the value needs to change. |
| `string / number / boolean` | The three most fundamental data types in JavaScript. |
| `function / =>` | Create reusable blocks of code with parameters and return values. |
| `if / else if / else` | Make decisions: run different code depending on conditions. |
| `for loop` | Repeat code a set number of times, with optional early exit via break. |

## Reflection questions

Before your next session, write a short answer to each of these:

- What is the difference between let and const? When would you use each?
- In your own words, what is a function and why is it useful?
- What would happen if you wrote the FizzBuzz conditions in the wrong order?
- Write one thing that confused you today and bring it to the next class.

## What is coming next?

In upcoming sessions you will learn:

- Arrays — storing and working with lists of data
- Objects — grouping related data and functions together
- DOM manipulation — using JavaScript to change what is on a web page
- Events — making pages respond to user actions like clicks and keypresses
- Fetch API — loading data from the internet without reloading the page

**Final note**

The most important thing you can do after this session is keep typing code. Even 20 minutes a day makes a significant difference. Every professional developer started exactly where you are now.

If you get stuck, remember: error messages are not failures — they are JavaScript telling you exactly what it needs. Read them carefully.